

Porcentaje: 40% Profesor: M.Sc. Jose Danilo Chinchilla Cruz	Puntos	Porcentaje	Nota
Instrucciones generales			
Esta evaluación está incluido dentro de las evaluaciones de la Directriz sobre Honestidad Académica , presentada y aceptada en el Programa del Curso , el incumplimiento con la directriz mencionada generará la aplicación correspondiente del artículo 31 del reglamento estudiantil vigente. A la hora de entregar el archivo se debe subir el proyecto completo a la sección correspondiente en el campus virtual, el nombre del archivo debe tener el siguiente formato. ProyectoFinal_Grupo#Horario			

Sistema Cliente/Servidor Concurrente para la Gestión y Monitoreo de Entregas – QuickDelivery S.A.

1. Contexto del Proyecto

La empresa de mensajería **QuickDelivery S.A.** enfrenta actualmente problemas de desincronización de información operativa. Los despachadores no cuentan con información en tiempo real sobre la ubicación de los vehículos ni el estado de los paquetes, ya que estos datos solo se conocen cuando el conductor regresa a la base.

Esta situación genera:

- Falta de información actualizada para los clientes finales.
- Dificultad para asignar nuevos paquetes a los vehículos más cercanos o disponibles.
- Riesgos en la trazabilidad de los envíos y en la seguridad de la información.
- Problemas de concurrencia al manejar múltiples repartidores simultáneamente.

2. Objetivo General del Proyecto

Desarrollar un **sistema distribuido Cliente/Servidor concurrente en Java**, que permita el monitoreo en tiempo real de vehículos y paquetes, utilizando programación orientada a objetos, multihilos, sockets, interfaces gráficas Swing y acceso seguro a base de datos mediante JDBC, aplicando buenas prácticas de diseño y arquitectura MVC.

3. Alcance General de la Solución

El sistema deberá permitir:

- Gestión de usuarios con distintos roles (administrador, despachador, conductor).
- Registro y administración de vehículos y paquetes.
- Comunicación Cliente/Servidor mediante sockets.
- Manejo concurrente de múltiples vehículos (cada vehículo como un hilo).
- Persistencia de la información en base de datos relacional.
- Registro de eventos y auditoría mediante archivos de log.
- Visualización gráfica del estado del sistema en tiempo real.
- Manejo adecuado de errores y validaciones.

4. Historias de Usuario (Requerimientos Funcionales)

El sistema deberá cubrir, como mínimo, las siguientes historias de usuario:

1. Gestión de usuarios y roles del sistema.
2. Autenticación y control de acceso según rol.
3. Registro, edición y eliminación de paquetes.
4. Registro y administración de vehículos utilizando herencia y polimorfismo.
5. Asignación de paquetes a vehículos disponibles.
6. Conexión de conductores al servidor mediante sockets.
7. Envío periódico de ubicación y estado del vehículo.
8. Actualización del estado de los paquetes (En tránsito, Entregado, Incidencia).
9. Visualización en tiempo real de vehículos y paquetes en una interfaz Swing.

10. Registro de incidencias durante las entregas.
11. Manejo robusto de errores de red, concurrencia y base de datos.
12. Auditoría y trazabilidad mediante archivos de log y base de datos.

5. Base de Datos

La solución deberá implementar una base de datos **normalizada**, con un mínimo de **10 tablas**, entre ellas:

- Usuarios
- Roles
- Vehículos
- Tipos de vehículo
- Conductores
- Paquetes
- Estados de paquete
- Asignaciones de paquetes
- Ubicaciones de vehículos
- Incidencias
- Logs del sistema

La conexión a la base de datos deberá realizarse mediante **JDBC**, aplicando buenas prácticas de seguridad como el uso de PreparedStatement.

6. Arquitectura y Tecnologías Obligatorias

El proyecto deberá evidenciar el uso de:

- Programación Orientada a Objetos (encapsulación, herencia, polimorfismo, interfaces).
- Manejo de excepciones y jerarquías de errores.
- Colecciones genéricas y colecciones concurrentes.
- Interfaces gráficas con Swing (JFrame, JTable, JDialog, etc.).
- Programación multihilo.
- Comunicación Cliente/Servidor mediante sockets.
- Persistencia en base de datos con JDBC.
- Arquitectura MVC.
- Manejo de archivos y serialización de objetos.

7. Entregables del Proyecto

El proyecto se divide en **tres entregables obligatorios**, con los siguientes pesos:

Entregable 1 – Análisis y Diseño (Semana 7 – 10%)

Contenido mínimo:

- Documento de requerimientos e historias de usuario.
- Diagrama de clases UML (uso de herencia e interfaces).
- Diagrama Entidad-Relación (DER) con al menos 10 tablas normalizadas.
- Prototipo de interfaces gráficas (mockups en Swing).

Rúbrica Entregable 1: Análisis y Diseño (Semana 7 – 10%)

Objetivo: Evaluar la capacidad de abstracción y modelado de la solución.

Criterio	Excelente (100-90)	Bueno (89-80)	Regular (79-70)	Deficiente (69-0)
Requerimientos e Historias de Usuario	Define 10+ historias claras con criterios de aceptación. Resuelven la problemática de logística.	Define 7-9 historias. Algunas son ambiguas pero cubren la funcionalidad básica.	Historias incompletas (menos de 5) o no alineadas al problema de la empresa.	No presenta historias de usuario o estas no tienen sentido con el proyecto.
Diagrama de Clases UML	Incluye herencia, polimorfismo e interfaces. Usa correctamente la simbología UML y el patrón MVC.	Incluye herencia y clases, pero omite interfaces o la relación con MVC es confusa.	Diagrama desordenado. No se aprecia el uso de polimorfismo o herencia requerida.	El diagrama es erróneo, incompleto o no refleja el código que se planea programar.
Diseño de Base de Datos (DER)	Presenta 10+ tablas normalizadas (3FN). Relaciones lógicas y tipos de datos correctos.	Presenta 8-9 tablas. Faltan detalles de normalización o algunas relaciones son débiles.	Menos de 7 tablas. Presenta redundancia de datos evidente y falta de integridad.	DER pobremente estructurado (menos de 5 tablas) o sin coherencia con el sistema.
Prototipo de Interfaz (Mockups)	Mockups detallados de Swing que muestran el flujo completo desde Login hasta el Monitor.	Mockups claros pero estáticos. No se visualiza cómo interactuará el usuario.	Capturas de pantalla o dibujos básicos que no reflejan la complejidad del sistema.	No presenta prototipos o estos no coinciden con las historias de usuario.

Entregable 2 – Producto Mínimo Viable (Semana 12 – 20%)

Contenido mínimo:

- Base de datos creada y funcional.
- Conexión a base de datos mediante JDBC.
- Servidor multihilo con sockets funcionales.
- Clientes vehículo conectándose concurrentemente.
- Interfaz Swing básica con CRUD de paquetes.

Rúbrica Entregable 2: Producto Mínimo Viable 80/20 (Semana 12 – 20%)

Objetivo: Evaluar la arquitectura de red, concurrencia y persistencia inicial.

Criterio	Excelente (100-90)	Bueno (89-80)	Regular (79-70)	Deficiente (69-0)
Comunicación Sockets	Servidor y clientes intercambian objetos/mensajes de forma estable y bidireccional.	Comunicación funcional pero presenta desconexiones ocasionales o errores de flujo.	Sockets solo envían mensajes simples de texto sin lógica de negocio aplicada.	No hay comunicación por red o el servidor no acepta conexiones múltiples.
Multihilos y Concurrencia	Implementa múltiples hilos (vehículos) funcionando simultáneamente sin bloqueos (Deadlocks).	Usa hilos pero se nota un manejo pobre de la sincronización de recursos compartidos.	Solo funciona un hilo a la vez o el sistema se congela al activar la concurrencia.	No implementa hilos o los hilos no realizan tareas útiles para la logística.
JDBC y CRUD	Conexión segura a DB. El CRUD de paquetes funciona al 100% usando PreparedStatement.	Conexión funcional pero el CRUD falla en ciertos casos (ej. borrado de datos).	Usa sentencias SQL inseguras (concatenación) o solo funciona la consulta (Select).	No logra conectar con la base de datos o el CRUD no es funcional.
Arquitectura MVC	Separación total de Modelo, Vista y Controlador. El código es limpio y legible.	Separación parcial. Hay lógica de negocio dentro de los eventos de la GUI (JFrame).	El código está mezclado. Es difícil distinguir entre la lógica y la interfaz.	No sigue ningún patrón de diseño. Todo el código está en una sola clase o paquete.

Entregable 3 – Sistema Completo y Defensa (Semana 14 – 10%)

Contenido mínimo:

- Manejo completo de excepciones.
- Registro de logs en archivos y base de datos.
- Uso de colecciones concurrentes.
- Seguridad básica en acceso a datos.
- Presentación y defensa del proyecto.

Rúbrica Entregable 3: Sistema Completo y Defensa (Semana 15 – 10%)

Objetivo: Evaluar la robustez, seguridad, optimización y dominio técnico.

Criterio	Excelente (100-90)	Bueno (89-80)	Regular (79-70)	Deficiente (69-0)
Manejo de Excepciones	Jerarquía de excepciones personalizada. El sistema no se cae ante entradas erróneas.	Manejo básico con Try-Catch generales. El sistema es estable pero poco informativo.	Pocas excepciones controladas. Errores comunes lanzan excepciones en consola.	No hay manejo de excepciones. El programa se cierra ante cualquier error.
Colecciones y Optimización	Uso eficiente de Colecciones Concurrentes (ConcurrentHashMap, etc.) para la flota.	Usa colecciones genéricas (ArrayList) pero sin considerar la concurrencia.	Colecciones mal implementadas que causan lentitud o errores de índice.	No utiliza colecciones; usa arreglos estáticos para datos dinámicos.
Persistencia (Logs y Serialización)	Registra auditoría en archivos de texto y serializa objetos de configuración con éxito.	Genera logs básicos pero no utiliza serialización de objetos como pide el curso.	Archivos de log desordenados o difíciles de leer. No hay persistencia local.	No hay manejo de archivos ni serialización de ningún tipo.
Defensa y Dominio Técnico	El equipo demuestra dominio total. Responde preguntas técnicas sobre hilos y sockets con solidez.	Demuestran dominio pero dudan en conceptos avanzados de concurrencia o red.	Solo un integrante conoce el código. Poca claridad al explicar el funcionamiento.	No conocen cómo funciona su propio código o no pueden explicar la lógica.

8. Modalidad de Trabajo

- Proyecto **grupal**, máximo **4 integrantes**.
- Uso obligatorio de **control de versiones**.
- Todos los integrantes deben aportar código y participar en la defensa.

9. Criterio Final de Evaluación

El proyecto será evaluado según:

- Cumplimiento de requerimientos.
- Calidad técnica de la solución.
- Aplicación de los contenidos del curso.
- Buenas prácticas de programación.
- Claridad y dominio en la presentación final.

Este proyecto representa una simulación de un problema real de una empresa y constituye la evaluación integradora final del curso.